DATABASE SYSTEMS

MongoDB

A Complete Guide

Introduction · Necessity · Terminology · Data Types · Query Language

Undergraduate Database Systems | NoSQL Module

Course Agenda

01

Introduction to MongoDB

What it is, history, architecture

02

Necessity of MongoDB

Why NoSQL? RDBMS limitations, use cases

03

MongoDB vs RDBMS Terms

Database, Collection, Document, Field comparison

04

Data Types in MongoDB

BSON types, scalars, arrays, embedded docs

05

MongoDB Query Language (MQL)

CRUD, operators, aggregation, SQL comparison

SECTION 01

Introduction to MongoDB

What it is, where it came from, and how it works

What is MongoDB?

INTRO

MongoDB is an open-source, cross-platform, document-oriented NoSQL database that stores data as flexible JSON-like documents instead of rows and columns.

Founded

2007 by Dwight Merriman, Eliot Horowitz & Kevin Ryan. Open-sourced in 2009. Name from 'humongous' — designed for massive scale.

Popularity

#5 most popular database worldwide. Used by Toyota, Forbes, eBay, Adobe, Bosch, SEGA and thousands more companies globally.

Document Model

Data stored as BSON (Binary JSON) documents — rich, nested structures with arrays. No rigid schema required.

Open Source

Free Community Edition. MongoDB Atlas offers fully managed cloud hosting on AWS, Azure, and GCP.

Performance

Horizontal scaling via sharding. In-memory operations for hot data. Indexes on any field including nested documents.

Ecosystem

MongoDB Compass (GUI), Atlas Charts, MongoDB BI Connector, Mongoose (Node.js ODM), and official drivers for 10+ languages.

The Document Model

INTRO

In MongoDB, a record is a Document — a rich JSON-like structure. A collection of documents replaces a database table. Related data is stored together.

```
// A MongoDB "User" document
{
  "_id"     : ObjectId("64a1f2b3c4d5e6f7a8b9c0d1"),
  "name"    : "Priya Sharma",
  "email"   : "priya@example.com",
  "age"     : 28,
  "active"  : true,
  "tags"    : ["student", "developer"],
  "address" : {
     "city"    : "Bangalore",
     "pin"     : "560001",
     "state"   : "Karnataka"
  },
  "courses"  : [
     { "code":"CS101", "grade":"A" },
     { "code":"MA201", "grade":"B+" }
  ],
  "createdAt": ISODate("2024-01-15")
}
```

- Auto-generated unique ID
- String field
- Number, Boolean types
- Array of values
- Embedded sub-document
- Array of documents

✅ No JOIN needed — address & courses live inside the user document!

MongoDB Architecture

INTRO

Cluster

A MongoDB deployment: one or more servers working together. Can be on-premise or cloud (Atlas).

Replica Set

3+ nodes: 1 Primary (reads/writes) + 2+ Secondaries (replicas). Automatic failover if primary goes down.

Shards

Horizontal partitioning. Each shard holds a subset of data. Shard key determines routing.

Database

Logical grouping of collections (similar to a schema in RDBMS). A cluster can hold many databases.

Collection

A group of MongoDB documents — analogous to a table. No fixed schema; documents can have different fields.

Document

The basic unit of data. A BSON object with key-value pairs. Maximum size: 16 MB.

SECTION 02

Necessity of MongoDB

Why traditional RDBMS falls short in modern applications

Limitations of Traditional RDBMS

NECESSITY

Relational databases are excellent for structured data and transactions, but modern applications generate data that does not fit the relational model well.

🔒 Rigid Schema

Every row must follow the same columns. Adding a new field requires ALTER TABLE — expensive on billions of rows. Cannot easily store optional or variable fields.

🔗 JOIN Complexity

Normalisation spreads data across many tables. Complex queries with 10+ JOINs become slow and hard to maintain. Performance degrades at scale.

📈 Vertical Scaling Only

Traditional RDBMS scales UP (more RAM, faster CPU, bigger disk). Horizontal scaling (adding nodes) is difficult — sharding is complex and limited.

🐌 Schema Migration Pain

Changing a table schema in production requires downtime or complex migration scripts. Agile development needs frequent schema changes.

🗂️ Unstructured Data

JSON, XML, multimedia, logs, sensor data do not map cleanly to tables. Forcing them into RDBMS requires workarounds like JSON columns or EAV pattern.

💸 Cost at Big Data Scale

Oracle, SQL Server licences are expensive at scale. At billions of rows, even open-source RDBMS require significant infrastructure investment.

Why MongoDB? — What Problems It Solves

NECESSITY

2× faster
than RDBMS on high-cardinality data

100 TB+
single cluster can manage seamlessly

16 MB
max document size (fits most use cases)

50M+
downloads of MongoDB Community

Flexible Schema

Documents in the same collection can have different fields. Add new fields to some documents without touching others. Perfect for evolving product catalogues, user profiles, configuration data.

Horizontal Scaling

Auto-sharding distributes data across nodes automatically. Add commodity servers to scale out. No single point of failure. Handles millions of writes per second across a cluster.

Rich Query Language

MQL supports filtering, projection, aggregation, full-text search, geospatial queries, and graph traversal — all on the document model without JOINs.

High Availability

Replica sets provide automatic failover in < 5 seconds. Read preference lets you route read traffic to secondaries, reducing load on the primary.

Real-World Use Cases Where MongoDB Excels

NECESSITY

E-Commerce Catalogs

Products have vastly different attributes:
Shirt: {color, size, material}
Laptop: {RAM, CPU, screen_size}
With RDBMS: hundreds of nullable columns or messy EAV. MongoDB: each product document has exactly the fields it needs.

User Profiles & Prefs

Social platforms store user preferences, activity history, connections, and settings. These vary enormously per user and evolve constantly. MongoDB stores it all in one document — one query retrieves the entire profile.

Real-Time Analytics

IoT sensors, clickstream events, and application logs produce millions of writes per second. MongoDB's write throughput and time-series collections are optimised for high-speed, time-stamped data ingestion.

Gaming Leaderboards

Game state, player inventory, session data, and matchmaking all change rapidly. MongoDB's in-memory performance and flexible schema handle dynamic game data without migrations.

Content Management

Blogs, news sites, and CMS platforms have articles with different fields (video, gallery, poll, review). MongoDB stores each content type naturally. Atlas Search adds full-text search with zero extra infrastructure.

Financial Operations

Account records, transaction history, fraud signals and risk scores per customer all combined in rich documents. Multi-document ACID transactions (MongoDB 4.0+) handle financial consistency requirements.

SECTION 03

MongoDB vs RDBMS Terminology

Mapping familiar SQL concepts to MongoDB equivalents

Terminology: RDBMS vs MongoDB

TERMINOLOGY

MongoDB uses a different vocabulary to describe its data model. Every concept you know from SQL has a MongoDB equivalent — but with important differences.

RDBMS Term	MongoDB Equivalent	Key Difference
Database	Database	Same name — but MongoDB databases are created lazily (no CREATE DATABASE needed)
Table	Collection	No fixed schema. Documents in a collection can have different fields and types
Row / Record	Document	Documents are JSON-like (BSON). Can be nested and contain arrays — rows cannot
Column	Field	Fields are per-document, not per-collection. Optional fields simply don't appear in the document
Primary Key	_id field	_id is auto-created as ObjectId if not specified. Must be unique per collection
Foreign Key	Reference / Embed	No enforced FK constraints. Either embed related data or store a reference (_id)
JOIN	$lookup / Embedding	Use embedded documents instead of JOINs when possible. $lookup for reference-based joins
INDEX	Index	Works the same — createIndex() on any field, including nested fields and arrays
VIEW	View	MongoDB has read-only views defined as aggregation pipelines
Stored Procedure	Map-Reduce / Aggregation	Aggregation Pipeline replaces stored procedures for complex data processing

Visual Structure Comparison

TERMINOLOGY

Side-by-side: how the same 'users' data is represented in a relational database vs MongoDB

RDBMS — Two Normalised Tables

users

user_id (PK)	name	email
1	Priya	priya@mail.com
2	Rohan	rohan@mail.com

addresses (FK → users.user_id)

addr_id	user_id (FK)	city	state
A1	1	Bangalore	Karnataka
A2	2	Mumbai	Maharashtra

⚠ *Need a JOIN to get user + address!*

```
SELECT u.name, a.city
FROM users u
JOIN addresses a ON u.user_id = a.user_id;
```

MongoDB — One Collection

```
// users collection — doc 1
{
  "_id": 1, "name": "Priya",
  "email": "priya@mail.com",
  "address": {
    "city": "Bangalore",
    "state": "Karnataka"
  }
}
// query — no JOIN needed!
db.users.findOne({"_id":1})
```

✅ **Single query retrieves user + address together!**

SECTION 04

Data Types in MongoDB

BSON types — scalar, complex, and special

What is BSON?

DATA TYPES

BSON (Binary JSON) is the serialisation format MongoDB uses to store documents. It extends JSON with additional data types like Date, ObjectId, Binary, and Decimal128 — and is more efficient to parse.

JSON

- Human-readable text format
- Only: string, number, boolean, null, array, object
- No native Date type (stored as string)
- No binary data support
- Parsed character by character — slower

BSON

- Binary encoded — compact and fast
- All JSON types PLUS: Date, ObjectId, Int32/64, Decimal128, Binary, Regex, Timestamp, MinKey/MaxKey
- Native Date type with millisecond precision
- Binary data support
- Length-prefixed — fast to traverse without full parse

BSON adds type information to every value — MongoDB knows whether a field is an integer, float, or date without any hints from the developer.

Scalar Data Types

DATA TYPES

String

```
"name": "Priya Sharma"
```

UTF-8 encoded. Most common type. Used for names, emails, codes.

Integer

```
"age": 28  // Int32
"views": NumberLong(1500000)
```

Int32 (32-bit) or Int64 (64-bit). Use NumberLong() for large integers.

Double

```
"price": 49.99  // 64-bit float
```

64-bit IEEE 754 float. Default for numeric literals in MQL.

Decimal128

```
"amount": NumberDecimal("9999.99")
```

High-precision 128-bit decimal. Use for financial values. Avoids float rounding errors.

Boolean

```
"active": true
"deleted": false
```

true or false. Same as JSON boolean.

Null

```
"middleName": null
```

Explicit null value. A field can be absent OR null — both are different.

ObjectId

```
"_id": ObjectId("64a1f2b3c4d5...")
```

12-byte unique identifier. Encodes timestamp + machine ID + process ID + counter.

Date

```
"joined": ISODate("2024-01-15")
```

Milliseconds since Unix epoch. Always stored in UTC. Use ISODate() constructor.

Complex & Special Data Types

DATA TYPES

Array

```
// Array of strings
"tags": ["mongodb","nosql","db"]
// Array of numbers
"scores": [95, 87, 92, 78]
// Array of documents
"courses": [
  { "code":"CS101","grade":"A" },
  { "code":"MA201","grade":"B" }
]
```

Arrays can hold ANY type — strings, numbers, objects, even other arrays (up to any depth). MongoDB can index and query individual array elements.

Embedded Document

```
// Nested object (sub-document)
"address": {
  "street" : "12 MG Road",
  "city"   : "Bangalore",
  "pin"    : "560001",
  "coords" : {
     "lat": 12.9716,
     "lon": 77.5946
  }
}
```

Embedded documents can be nested up to 100 levels deep. Query with dot notation: db.users.find({'address.city':'Bangalore'})

Binary Data

`BinData(0, "aGVsbG8=")`

Store images, files, encrypted data. Up to 16MB per document.

Regex

`/^priya/i`

Store PCRE regular expressions directly as a BSON type.

Timestamp

`Timestamp(1705000000, 1)`

Internal MongoDB type for replication ordering. 64-bit value.

MinKey/MaxKey

`MinKey(), MaxKey()`

Sentinel values — compare less/greater than ALL other BSON values. Used in sharding.

SECTION 05

MongoDB Query Language (MQL)

CRUD operations, operators, and aggregation pipeline

CRUD Operations Overview

MQL

MQL (MongoDB Query Language) is the API for interacting with MongoDB. All operations follow a consistent pattern: db.collectionName.methodName(filter, options)

➕ CREATE

SQL equivalent:

`INSERT INTO`

Methods:

- `insertOne()`
- `insertMany()`

🔍 READ

SQL equivalent:

`SELECT`

Methods:

- `find()`
- `findOne()`
- `countDocuments()`

✏️ UPDATE

SQL equivalent:

`UPDATE`

Methods:

- `updateOne()`
- `updateMany()`
- `replaceOne()`

🗑️ DELETE

SQL equivalent:

`DELETE`

Methods:

- `deleteOne()`
- `deleteMany()`

INSERT Operations — insertOne & insertMany

MQL

insertOne(document) — insert a single document

```
db.students.insertOne({
  "name"    : "Anjali Mehta",
  "rollNo"  : "CS2024001",
  "branch"  : "Computer Science",
  "cgpa"    : 8.9,
  "active"  : true
});
```

Response:

```
{
  acknowledged: true,
  insertedId: ObjectId(
    "64a1f2b3c4d5e6f7...")
}
```

insertMany([doc1, doc2, ...]) — insert multiple documents in one call

```
db.students.insertMany([
  { "name":"Rohan", "branch":"ECE", "cgpa":7.5 },
  { "name":"Sneha", "branch":"CSE", "cgpa":9.1, "scholarship": true },
  { "name":"Kiran", "branch":"ME",  "cgpa":8.0 }
],  { ordered: false }  // continue even if one fails
);
```

READ Operations — find, findOne & Projections

MQL

```
// Syntax:  db.collection.find( <filter> , <projection> )
db.students.find({})                              // all documents (like SELECT * FROM students)
db.students.findOne({"branch":"CSE"})             // first matching document only
db.students.find({"branch":"CSE"}).limit(5)       // only 5 results
```

Projections — control which fields are returned

```
// 1 = include,  0 = exclude  (cannot mix except for _id)
db.students.find(                    // SQL: SELECT name, cgpa FROM students
   { "branch": "CSE" },              //       WHERE branch = "CSE"
   { "name":1, "cgpa":1, "_id":0 } // projection: only name and cgpa, hide _id
);
```

Useful Cursor Methods (chain them together)

`.sort({'cgpa':-1})`	Sort DESC by cgpa (1=ASC, -1=DESC)
`.limit(10)`	Return at most 10 documents
`.skip(20)`	Skip first 20 (for pagination)
`.count()` / `countDocuments({})`	Count matching documents

Query Operators — Comparison & Logical

MQL

Comparison Operators

Operator	Meaning	Example
$eq	Equal to	{"cgpa":{"$eq":9.0}}
$ne	Not equal	{"branch":{"$ne":"ECE"}}
$gt	Greater than	{"cgpa":{"$gt":8.0}}
$gte	Greater than or equal	{"cgpa":{"$gte":8.0}}
$lt	Less than	{"age":{"$lt":20}}
$lte	Less than or equal	{"age":{"$lte":22}}
$in	Matches any in array	{"branch":{"$in":["CSE","IT"]}}
$nin	Matches none in array	{"grade":{"$nin":["F","D"]}}

Logical Operators

```
// $and — both conditions true
db.students.find({
  "$and": [
    { "cgpa":   { "$gte": 8.0 } },
    { "branch": "CSE" }
  ]
});
// $or — at least one true
db.students.find({
  "$or": [
    { "scholarship": true },
    { "cgpa": { "$gte": 9.0 } }
  ]
});
// $not   $nor   $exists   $type
```

$exists — field exists **$type** — field BSON type **$elemMatch** — match array element **$size** — array length **$regex** — pattern match

UPDATE Operations — updateOne, updateMany & Operators

MQL

Syntax: db.collection.updateOne(<filter>, <update>, <options>)

```
// Update one student's CGPA
db.students.updateOne(
  { "rollNo": "CS2024001" },   // filter
  {
    "$set":  { "cgpa": 9.2 },  // update field
    "$inc":  { "credits": 4 }, // increment by 4
    "$push": { "tags":"topper" } // add to array
  }
);
```

```
// Bulk update — all CSE students
db.students.updateMany(
  { "branch": "CSE" },
  {
    "$set": {
      "eligible": true,
      "dept": "CS&IT"
    }
  }
);
```

Common Update Operators

Operator	Effect	Array?
$set	Set field to a value	No
$unset	Remove a field from document	No
$inc	Increment a numeric field by n	No
$mul	Multiply a field by n	No
$rename	Rename a field	No

DELETE Operations & Upsert

MQL

Syntax: db.collection.deleteOne(<filter>) / db.collection.deleteMany(<filter>)

```
// Delete one document
db.students.deleteOne({
  "rollNo": "CS2024001"
});
// Delete all inactive students
db.students.deleteMany({
  "active": false
});
```

Safe DELETE tips:

- Always run find() first to confirm your filter
- Use deleteOne() not deleteMany() unless intentional
- MongoDB has no TRUNCATE — use drop() to remove entire collection
- Deletions are permanent — no built-in recycle bin

Upsert — update if exists, insert if not (combines UPDATE + INSERT)

```
// upsert: true – creates doc if no match
db.students.updateOne(
  { "rollNo": "CS2025001" },
  {
    "$set": { "name":"Vikram", "cgpa":8.5 }
  },
  { "upsert": true }    // <-- magic option
);
```

Upsert behaviour:

If rollNo CS2025001 EXISTS: update cgpa and name fields.
If rollNo CS2025001 DOES NOT exist: creates a brand new document with those fields.

This is equivalent to:
INSERT INTO ... ON DUPLICATE KEY UPDATE ...
in MySQL.

Aggregation Pipeline

MQL

The Aggregation Pipeline transforms documents through a sequence of stages. Each stage receives documents from the previous stage — like a UNIX pipe for data.




```
// Find average CGPA per branch — only branches with avg >= 8.0, sorted desc
db.students.aggregate([
  { "$match":   { "active": true } },
  { "$group":   { "_id": "$branch",
                  "avgCGPA": { "$avg": "$cgpa" },
                  "totalStudents": { "$sum": 1 } } },
  { "$match":   { "avgCGPA": { "$gte": 8.0 } } },
  { "$project": { "branch":"$_id", "avgCGPA":1, "totalStudents":1, "_id":0 } },
  { "$sort":    { "avgCGPA": -1 } },
  { "$limit":   5 }
]);
```

SQL vs MQL — Side-by-Side Comparison

MQL

Every SQL concept has a direct MQL equivalent. The biggest difference: MQL works on documents, not rows.

SQL	MQL (MongoDB)
`SELECT * FROM students`	`db.students.find({})`
`SELECT name, cgpa FROM students WHERE branch="CSE"`	`db.students.find({"branch":"CSE"},{"name":1,"cgpa":1})`
`SELECT * FROM students WHERE cgpa > 8 ORDER BY cgpa DESC LIMIT 10`	`db.students.find({"cgpa":{"$gt":8}}).sort({"cgpa":-1}).limit(10)`
`SELECT COUNT(*) FROM students WHERE active=1`	`db.students.countDocuments({"active":true})`
`SELECT branch, AVG(cgpa) FROM students GROUP BY branch`	`db.students.aggregate([{"$group":{"_id":"$branch","avg":{"$avg":"$cgpa"}}}])`
`INSERT INTO students(name,cgpa) VALUES("Ali",8.5)`	`db.students.insertOne({"name":"Ali","cgpa":8.5})`
`UPDATE students SET cgpa=9.0 WHERE rollNo="CS001"`	`db.students.updateOne({"rollNo":"CS001"},{"$set":{"cgpa":9.0}})`
`DELETE FROM students WHERE active=0`	`db.students.deleteMany({"active":false})`

Key Takeaways

01

MongoDB is document-oriented

Stores rich JSON-like BSON documents. No rigid schema. Related data can be embedded in a single document — fewer JOINs.

02

NoSQL fills RDBMS gaps

Horizontal scaling, flexible schema, unstructured data, agile development — all areas where RDBMS struggles.

03

Familiar but different terms

Table→Collection, Row→Document, Column→Field, PK→_id. Same concepts, richer implementation.

04

BSON extends JSON

ObjectId, Date, Int32/64, Decimal128, Binary, Regex — all native BSON types. Type-safe without a schema enforcer.

05

MQL is expressive

Rich operators: $gt, $in, $or, $regex. Update operators: $set, $push, $pull. Aggregation pipeline for SQL-like GROUP BY.